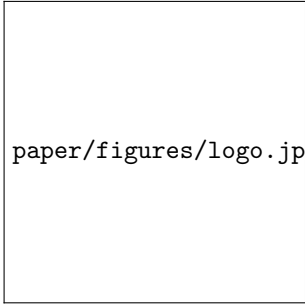




paper/figures/logo.jpg

**Sapienza University of Rome**

Department of Computer, Control, and Management Engineering  
Master of Science in Data Science

MASTER'S THESIS

# **Ambiguity-Aware Classification of Cloud Computing Requirements Using BERT and Large Language Models**

Thesis Advisor  
**Prof. [Advisor Name]**

Candidate  
**Claudio Giannini**

**Academic Year 2025–2026**

*[ Optional quote here.]*

## **Abstract**

[Abstract to be written.]

Keywords: requirements engineering, ambiguity, BERT, cloud computing, large language models, natural language processing.



# Contents

|                                                                                  |           |
|----------------------------------------------------------------------------------|-----------|
| <b>List of Figures</b>                                                           | <b>v</b>  |
| <b>List of Tables</b>                                                            | <b>vi</b> |
| <b>1 Introduction</b>                                                            | <b>1</b>  |
| <b>2 Literature Review</b>                                                       | <b>3</b>  |
| 2.1 The AI-CRAS Baseline: Cloud Requirement Classification . . . . .             | 3         |
| 2.2 Ambiguity in Natural Language Requirements . . . . .                         | 4         |
| 2.3 BERT and Transformer-Based Classification . . . . .                          | 4         |
| 2.4 Large Language Models for Disambiguation and Classification . . . . .        | 5         |
| <b>3 Dataset and Annotation</b>                                                  | <b>6</b>  |
| 3.1 The AI-CRAS Dataset . . . . .                                                | 6         |
| 3.2 Ambiguity Taxonomy . . . . .                                                 | 6         |
| 3.3 Annotation Procedure . . . . .                                               | 7         |
| 3.4 Dataset Statistics . . . . .                                                 | 7         |
| <b>4 Methodology</b>                                                             | <b>8</b>  |
| 4.1 BERT Classification Pipeline . . . . .                                       | 8         |
| 4.2 LLM-Based Disambiguation . . . . .                                           | 8         |
| 4.3 LLM as Direct Classifier . . . . .                                           | 8         |
| 4.3.1 Zero-Shot Classification . . . . .                                         | 8         |
| 4.3.2 Few-Shot Classification . . . . .                                          | 8         |
| 4.4 Evaluation Metrics . . . . .                                                 | 8         |
| <b>5 Experimental Evaluation</b>                                                 | <b>10</b> |
| 5.1 Experimental Setup . . . . .                                                 | 10        |
| 5.2 Baseline: BERT on the Full Dataset . . . . .                                 | 10        |
| 5.3 RQ1 — Effect of Ambiguity on BERT Classification . . . . .                   | 10        |
| 5.4 RQ2 — BERT on LLM-Disambiguated Sentences . . . . .                          | 10        |
| 5.5 RQ3 — LLM as Direct Classifier . . . . .                                     | 10        |
| 5.5.1 Zero-Shot Setting . . . . .                                                | 11        |
| 5.5.2 Few-Shot Setting . . . . .                                                 | 11        |
| 5.6 Cross-Experiment Comparison . . . . .                                        | 11        |
| <b>6 Discussion and Conclusion</b>                                               | <b>12</b> |
| 6.1 Interpretation of Results . . . . .                                          | 12        |
| 6.2 Answers to Research Questions . . . . .                                      | 12        |
| 6.2.1 RQ1: Does ambiguity negatively affect BERT classification performance? . . | 12        |
| 6.2.2 RQ2: Can LLM disambiguation improve BERT classification accuracy? . .      | 12        |
| 6.2.3 RQ3: Can LLMs match or exceed BERT as direct classifiers? . . . . .        | 12        |

|     |                                               |           |
|-----|-----------------------------------------------|-----------|
| 6.3 | Threats to Validity and Limitations . . . . . | 12        |
| 6.4 | Future Work . . . . .                         | 12        |
| 6.5 | Conclusion . . . . .                          | 12        |
|     | <b>Bibliography</b>                           | <b>13</b> |
|     | <b>A Appendix</b>                             | <b>14</b> |

# List of Figures

# List of Tables



# Chapter 1

## Introduction

The classification of cloud computing requirements from natural language documents is a well-studied problem in requirements engineering. Casalicchio and Cotumaccio’s AI-CRAS framework [3] addressed it by fine-tuning a BERT model on a domain-specific dataset of cloud requirement sentences, achieving a recall of 0.96 and an F1-score of 0.92 on the binary task of identifying whether a sentence constitutes a requirement.

What AI-CRAS does not examine is the effect of *ambiguity*. Many requirements are phrased in ways that allow more than one interpretation — due to vague wording, grammatical structure, or missing context. Natural language requirements can exhibit several distinct forms of ambiguity: lexical, syntactic, semantic, pragmatic, and language-error (described in detail in Chapter 3). These are common in real-world requirement documents, but their effect on model performance has not been studied.

This thesis tests whether ambiguity matters for classification — and whether certain categories are more vulnerable to it than others. Using the same AI-CRAS dataset, each sentence is manually annotated with ambiguity labels. Four experiments are then run: BERT on the full original dataset as a baseline, BERT restricted to the unambiguous subset, BERT on LLM-rewritten versions of the ambiguous sentences, and an LLM used directly as a classifier in both zero-shot and few-shot settings. For each experiment, results are broken down by the six cloud service categories to assess whether ambiguity affects them uniformly.

The three research questions are:

- **RQ1:** Does ambiguity negatively affect BERT classification performance, and which of the six cloud service categories is most affected?
- **RQ2:** Can LLM-based rewriting of ambiguous sentences improve BERT’s accuracy — both overall and for the most ambiguity-sensitive categories?
- **RQ3:** Can an LLM used directly as a classifier match or exceed BERT?

The main contributions of this thesis are: an ambiguity-annotated extension of the AI-CRAS dataset; an empirical analysis of how ambiguity affects BERT-based classification, including per-category breakdowns; and a comparative evaluation of LLMs as both a disambiguation tool and a direct classifier.

The remainder of this thesis is structured as follows. Chapter 2 reviews the literature. Chapter 3 describes the dataset and the annotation procedure. Chapter 4 covers the methodology. Chapter 5 presents the experimental results. Chapter 6 discusses the findings and concludes.

## Chapter 2

# Literature Review

### 2.1 The AI-CRAS Baseline: Cloud Requirement Classification

The AI-CRAS framework [3] proposes an AI-driven methodology for automating the analysis and specification of cloud computing requirements expressed in natural language. It addresses a recognised gap in the literature: while substantial research exists on NLP-assisted requirements engineering for generic software systems and for security requirements, little work targets the cloud domain specifically, where requirements span a heterogeneous mix of functional and non-functional concerns across IaaS, PaaS, and SaaS service models.

The methodology is structured into six phases: definition of a cloud service ontology, fine-tuning of a BERT-based classifier, automatic extraction of key sentences from requirement documents, classification of sentences as requirement or non-requirement, mapping of requirements to cloud services using the ontology, and automatic production of a structured specification document. The cloud service ontology organises requirements hierarchically into six main categories — Compute, Data Handling, Network, Security & Compliance, Management & Monitoring, and Cloud Service Essentials — each comprising three to five subcategories and a total of 68 cloud features at the lowest level.

The dataset was constructed from public Request for Proposal (RFP) documents and seven synthetically generated documents produced using GPT-4 to enrich the training data. Each of the resulting 2,164 sentences was independently labelled by two annotators with one-hot encoded assignments to the six ontology categories; disagreements led to exclusion of the disputed sample. Requirements account for 68.1% of the entries.

The classification model fine-tunes BERT<sub>BASE-UNCASED</sub> with a dropout layer (rate 0.3) and a linear output layer mapping the 768-dimensional pooled representation to six binary logits, trained with binary cross-entropy loss and the Adam optimizer. The hyperparameter configuration uses a maximum sequence length of 300 tokens, batch size of 16, learning rate of  $3 \times 10^{-5}$ , weight decay of  $10^{-6}$ , and a classification threshold of 0.20. On an 85/15 train-validation split, the model achieved a binary recall of 0.96 and an F1-score of 0.92, and a multi-label macro-averaged F1-score of 0.76, with a micro- and macro-averaged ROC-AUC of 0.96.

These results establish AI-CRAS as the baseline for automated cloud requirement classification and for the experiments in this thesis. The framework, however, operates under an implicit assumption that its input sentences are clear and well-formed — an assumption that rarely holds

in real-world procurement documents, where requirements are routinely vague, structurally ambiguous, or context-dependent. Whether such ambiguity degrades classification performance is the question that motivates the remainder of this review.

## 2.2 Ambiguity in Natural Language Requirements

If ambiguity matters for classification, it is necessary to define what it is and what forms it takes. The difficulty of poorly specified requirements is not simply a matter of coverage: a requirement can be correctly scoped and still be ambiguous — open to more than one valid interpretation. An ambiguous requirement may be correctly identified *as* a requirement by a classifier while still being fundamentally misunderstood by the system that acts on it. Bano [1] defines ambiguity in requirements engineering as “having multiple interpretations despite the reader’s knowledge of the RE context”, and identifies it as more intractable than other defects such as incompleteness.

Bano’s work also provides the structured taxonomy — building on Berry’s widely adopted classification — that this thesis uses to annotate and analyse ambiguous sentences:

Lexical ambiguity of a word arises when a word can have several different meanings. Syntactic ambiguity or structural ambiguity in a sentence arises when the sentence can be parsed in more than one way, resulting in more than one grammatical structure where each provides different meanings. Semantic ambiguity arises in a sentence when predicate logic of a sentence can lead to multiple interpretations, although there is no lexical, syntactic, or structural ambiguity. Language errors ambiguity is a grammatically wrong construction that is nevertheless understood and possibly in different ways due to the error. Pragmatic ambiguity is concerned with the relationship between the interpretations of a sentence and its context. [1]

These five categories — lexical, syntactic, semantic, language-error, and pragmatic ambiguity — are not rare edge cases. Bano’s mapping study of the RE literature found that 81% of empirical work on requirements ambiguity focuses on detection alone, with very little attention to resolution. This thesis directly addresses that gap: rather than detecting ambiguity, it tests whether automated resolution — rewriting ambiguous sentences with an LLM — can improve downstream classification. The annotation procedure that operationalises this taxonomy is described in Chapter 3.

## 2.3 BERT and Transformer-Based Classification

To measure the effect of ambiguity on classification, a model is needed that has been shown to work well on this specific task. BERT [4] — the same model family used by AI-CRAS — is such a model. It is built on the transformer architecture [6], which replaced recurrent networks with a self-attention mechanism that allows every token in a sequence to attend to all others simultaneously, producing richer contextual representations than its predecessors. BERT extends this foundation by pre-training on large text corpora using masked language modelling and next-sentence prediction, yielding representations that can be fine-tuned for downstream tasks with relatively little labelled data.

Fine-tuned BERT classifiers have demonstrated strong performance across a range of requirements engineering tasks: identifying requirements from unstructured documents, classifying functional and non-functional requirements, and detecting security-related statements. AI-CRAS’s results on cloud requirement classification — 0.92 F1-score on the binary task — confirm that this architecture is well-suited to the domain and the task. Using the same model family is also a deliberate methodological choice: it ensures that the results in this thesis are directly comparable to AI-CRAS’s published baseline, so that any performance differences can be attributed to ambiguity rather than to a change in model.

## 2.4 Large Language Models for Disambiguation and Classification

If ambiguity degrades BERT’s performance, the natural next question is whether it can be fixed. Large language models offer two distinct avenues for doing so.

The GPT family [5] introduced a paradigm shift: rather than fine-tuning on labelled data for each task, these models are prompted with natural language instructions and respond accordingly. This makes them natural candidates for requirement disambiguation — given an ambiguous sentence, an LLM can be asked to rewrite it in clearer terms while preserving the original intent, without requiring any task-specific training. This thesis tests whether feeding such rewritten sentences back to the BERT classifier (RQ2) recovers the performance lost to ambiguity.

Beyond rewriting, LLMs can also perform classification directly through prompting, bypassing BERT entirely. Brown et al. demonstrated that GPT-3 can perform few-shot classification by conditioning on a small number of labelled examples provided in the prompt [2], and Wei et al. showed that instruction-tuned models generalise to unseen classification tasks in zero-shot settings [7]. Whether this general capability extends to the specific and technical domain of cloud requirement classification — and whether it is competitive with a fine-tuned BERT model — is what RQ3 investigates.

## Chapter 3

# Dataset and Annotation

This chapter describes the dataset used in this work and the procedure adopted for its construction and enrichment. The experiments conducted in this thesis rely on the dataset introduced in the AI-CRAS framework, extended with manually annotated ambiguity labels.

### 3.1 The AI-CRAS Dataset

The original dataset was introduced by Casalicchio and Cotumaccio [3] to support the training and evaluation of machine learning models for the classification of cloud computing requirements. It was constructed by collecting sentences from publicly available Request for Proposal (RFP) documents related to cloud procurement, supplemented with examples generated using generative AI techniques. Each sentence was manually annotated according to a binary requirement label (requirement vs. non-requirement) and mapped to one of six cloud service categories: Compute, Data Handling, Network, Security Compliance, Management & Monitoring, and Cloud Service Essentials.

### 3.2 Ambiguity Taxonomy

The annotation of ambiguity in this work follows the taxonomy established in the requirements engineering literature, which identifies five distinct types of ambiguity that commonly appear in natural language requirements:

- **Lexical ambiguity:** a word or phrase carries multiple meanings depending on context (e.g., “high availability” may imply different service levels for different stakeholders).
- **Syntactic ambiguity:** the grammatical structure of a sentence admits more than one valid parse, leading to different readings.
- **Semantic ambiguity:** the logical content of a requirement can be interpreted differently by different stakeholders even when the syntax is unambiguous.
- **Pragmatic ambiguity:** the intended meaning relies on implicit contextual knowledge not present in the text itself.

- **Language-error ambiguity:** grammatical mistakes or typographical errors that obscure the intended meaning.

### 3.3 Annotation Procedure

In order to support the analysis of ambiguity, the original dataset was enriched with additional manually annotated columns. A binary column `is_ambiguous` was added to indicate whether a sentence contains any form of ambiguity. A further set of columns following the format `is_{type}_ambiguous` records the presence of each specific ambiguity category defined above.

All annotations were produced manually to ensure labeling quality and consistency. [Inter-annotator agreement details to be added if applicable.]

### 3.4 Dataset Statistics

[To be completed: total sentence count, proportion of ambiguous sentences, distribution across ambiguity types, class balance across cloud categories.]

# Chapter 4

## Methodology

This chapter describes the three methodological approaches used in the experimental evaluation: the BERT classification pipeline used as the primary classifier, the LLM-based disambiguation strategy applied to ambiguous requirements, and the LLM direct classification setup evaluated in zero-shot and few-shot configurations.

### 4.1 BERT Classification Pipeline

[To be completed: description of the BERT variant used, fine-tuning procedure, training data split, hyperparameters, hardware setup.]

### 4.2 LLM-Based Disambiguation

[To be completed: which LLM was used, prompting strategy, how semantic meaning was preserved, example prompt, post-processing of outputs.]

### 4.3 LLM as Direct Classifier

[To be completed: model selection rationale, prompt structure for classification.]

#### 4.3.1 Zero-Shot Classification

[To be completed: prompt design, how class labels were communicated to the model with no examples.]

#### 4.3.2 Few-Shot Classification

[To be completed: number of examples per class, selection strategy for examples, prompt structure.]

### 4.4 Evaluation Metrics

All experiments are evaluated using precision, recall, and F1-score. For the binary classification task (requirement vs. non-requirement) standard macro-averaged metrics are reported. For the



multi-label cloud category classification task, per-category and macro-averaged scores are reported to capture performance variation across categories.

## Chapter 5

# Experimental Evaluation

This chapter presents the results of the four experimental scenarios designed to answer the three research questions. Each experiment is described in terms of its setup, the results obtained, and a brief interpretation.

### 5.1 Experimental Setup

[To be completed: hardware, model versions (BERT variant, LLM), training hyperparameters, random seeds, dataset splits used across all experiments.]

### 5.2 Baseline: BERT on the Full Dataset

[To be completed: results of fine-tuned BERT on the complete original dataset, both binary classification and multi-label category classification. Establishes the performance reference for all subsequent comparisons.]

### 5.3 RQ1 — Effect of Ambiguity on BERT Classification

[To be completed: results of BERT trained and/or evaluated on the subset of unambiguous sentences. Comparison against baseline to assess whether removing ambiguous examples improves or changes performance. Breakdown by ambiguity type if applicable.]

### 5.4 RQ2 — BERT on LLM-Disambiguated Sentences

[To be completed: results of BERT evaluated on the version of the dataset where ambiguous sentences have been rewritten by the LLM. Comparison against baseline and RQ1 results.]

### 5.5 RQ3 — LLM as Direct Classifier

[To be completed: overall framing of the LLM-as-classifier evaluation and comparison with BERT results.]

### 5.5.1 Zero-Shot Setting

[To be completed: results of prompting the LLM to classify sentences with no examples provided. Performance on binary and multi-label tasks.]

### 5.5.2 Few-Shot Setting

[To be completed: results of prompting the LLM with a small number of labeled examples. Performance on binary and multi-label tasks. Comparison with zero-shot.]

## 5.6 Cross-Experiment Comparison

[To be completed: summary table comparing all four experimental scenarios across key metrics. Discussion of which approach performs best overall and which performs best per ambiguity type.]

# Chapter 6

## Discussion and Conclusion

### 6.1 Interpretation of Results

[To be completed: high-level reading of the experimental findings, unexpected patterns, and what the results suggest about the relationship between ambiguity and classification performance.]

### 6.2 Answers to Research Questions

#### 6.2.1 RQ1: Does ambiguity negatively affect BERT classification performance?

[To be completed.]

#### 6.2.2 RQ2: Can LLM disambiguation improve BERT classification accuracy?

[To be completed.]

#### 6.2.3 RQ3: Can LLMs match or exceed BERT as direct classifiers?

[To be completed.]

### 6.3 Threats to Validity and Limitations

[To be completed: dataset size, single-annotator labeling, LLM non-determinism, generalisability beyond cloud computing domain.]

### 6.4 Future Work

[To be completed: multi-annotator studies, other domains, fine-tuned LLM classifiers, automated ambiguity detection.]

### 6.5 Conclusion

[To be completed: brief summary of the thesis contributions and closing remarks.]

# Bibliography

- [1] Muneera Bano. “Addressing the Challenges of Requirements Ambiguity: A Review of Empirical Literature”. In: *2015 IEEE Fifth International Workshop on Empirical Requirements Engineering (EmpiRE)*. Ottawa, ON, Canada: IEEE, 2015, pp. 21–24.
- [2] Tom B. Brown et al. “Language Models are Few-Shot Learners”. In: *Advances in Neural Information Processing Systems* 33 (2020).
- [3] Emiliano Casalicchio and Alberto Cotumaccio. “AI-CRAS: AI-driven Cloud Service Requirement Analysis and Specification”. In: *2015 IEEE Fifth International Workshop on Empirical Requirements Engineering (EmpiRE)*. Ottawa, ON, Canada: IEEE, Aug. 2015. DOI: 10.1109/EmpIRE.2015.7431303.
- [4] Jacob Devlin et al. “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”. In: (2019). arXiv: 1810.04805.
- [5] OpenAI. *GPT-4 Technical Report*. Tech. rep. OpenAI, 2023.
- [6] Ashish Vaswani et al. “Attention Is All You Need”. In: *Advances in Neural Information Processing Systems* 30 (2017).
- [7] Jason Wei et al. “Finetuned Language Models Are Zero-Shot Learners”. In: *International Conference on Learning Representations* (2022).

## Appendix A

# Appendix

[Appendix content to be added.]

## Acknowledgements

[Acknowledgements to be written.]

Claudio Giannini, Rome, March 2026

